

Deep Neural Networks - Assignment1

Table of Contents

- [Dataset Selection](#)
- [Dataset Summary & Data Types](#)
- [Data Pre-Processing](#)
- [Baseline Linear Regression Model](#)
- [MLP Model \(From Scratch\)](#)
- [Model Training & Loss Curves](#)
- [Evaluation Metrics & Results](#)
- [Performance Comparison](#)
- [Overall Evaluation & Discussion](#)
- [Get Assignment Results](#)

GROUP 264

S.NO	NAME	BITS ID
1	ADURTI SAI VENKATESH	2024AC05968
2	ANKITA PRIYA	2024ac05907
3	D MALLIKARJUNA REDDY	2024AC05653
4	LIKHITHA NAVEEN	2024ac05916
5	PRANAV MEHROTRA	2024ac05914

Dataset Selection

**SOURCE: <https://www.kaggle.com/datasets/rohanrao/air-quality-data-in-india/versions/1?resource=download>

We use the “**Air Quality Data in India (2015–2020)**” dataset from Kaggle (public tabular data with daily city-level pollution readings).

- **Source:** Kaggle – *Air Quality Data in India* (author: Rohan Rao)
- **Samples:** More than 500 daily city–day records across multiple Indian cities
- **Features:** Multiple pollutant concentration features such as `PM2.5` , `PM10` , `NO2` , `CO` , `S02` , `O3` , `NH3` , `Benzene` , `To1uene` , `Xylene` and related metadata
- **Target variable:** `AQI` (Air Quality Index)

This is a **regression** problem because `AQI` is a continuous numeric value.

For evaluation, I use **RMSE (Root Mean Squared Error)** as the **primary metric** because:

- RMSE is in the same units as AQI, making prediction errors easy to interpret as “AQI points”, and
- it penalises larger errors more strongly than MAE, which is important in environmental monitoring, where large

mis-predictions (e.g., predicting a hazardous day as moderate) are particularly undesirable.

In addition to RMSE, I also report **MSE, MAE and R^2** as secondary regression metrics.

Load Dataset

```
In [33]: import pandas as pd

# Load dataset
file_path = "data/city_day.csv"
df = pd.read_csv(file_path)

print("Dataset Loaded Successfully")
print("File:", file_path)

print("\nShape (rows, columns):", df.shape)

print("\nColumn Names:")
print(df.columns.tolist())

print("\nPreview of Data:")
df.head()
```

Dataset Loaded Successfully
File: data/city_day.csv

Shape (rows, columns): (25555, 16)

Column Names:
['City', 'Date', 'PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'Ozone', 'Benzene', 'Toluene', 'Xylene', 'AQI', 'AQI_Bucket']

Preview of Data:

```
Out[33]:
```

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	Ozone	Benzene
0	Ahmedabad	2015-01-01	NaN	NaN	NaN	18.22	17.15	NaN	0.92	27.64	133.36	0.00
1	Ahmedabad	2015-01-02	NaN	NaN	NaN	15.69	16.46	NaN	0.97	24.55	34.06	3.68
2	Ahmedabad	2015-01-03	NaN	NaN	NaN	19.30	29.70	NaN	17.40	29.07	30.70	6.80
3	Ahmedabad	2015-01-04	NaN	NaN	NaN	18.48	17.97	NaN	1.70	18.59	36.08	4.43
4	Ahmedabad	2015-01-05	NaN	NaN	NaN	21.42	37.76	NaN	22.10	39.33	39.31	7.01

Dataset Summary & Data Types

The dataset contains only **tabular, non-image, non-text data**, satisfying the assignment requirement of using a real-world tabular dataset.

Key columns are:

- **Numeric pollutant features:** PM2.5 , PM10 , NO , NO2 , NOx , NH3 , CO , SO2 , O3 , Benzene , Toluene , Xylene
- **Target:** AQI (Air Quality Index, numeric)
- **Categorical / metadata columns:** City , AQI_Bucket , and a date/time column (Date)

For modelling:

- I treat **AQI** as a **continuous numeric target** for regression.
- I use **numeric pollutant concentration features** as inputs to the models.
- Non-numeric metadata such as **City** and **Date** are not used directly as features in this baseline experiment.

Thus, the final modelling table is a numeric feature matrix with **≥ 5 input features** and **one continuous target**, as required by the assignment.

```
In [34]: print("\nData Types:")
print(df.dtypes)

print("\nSummary Statistics (Numeric Columns):")
df.describe()
```

Data Types:

City	object
Date	object
PM2.5	float64
PM10	float64
NO	float64
NO2	float64
NOx	float64
NH3	float64
CO	float64
SO2	float64
Ozone	float64
Benzene	float64
Toluene	float64
Xylene	float64
AQI	float64
AQI_Bucket	object
dtype:	object

Summary Statistics (Numeric Columns):

Out[34]:

	PM2.5	PM10	NO	NO2	NOx	NH3	
count	21300.000000	14853.000000	21046.000000	22396.000000	21534.000000	15770.000000	23
mean	72.824121	127.653183	17.658008	29.481695	33.835887	25.667720	
std	67.356092	94.974574	22.539144	25.278898	32.688813	27.379991	
min	0.040000	0.010000	0.020000	0.010000	0.000000	0.010000	
25%	32.157500	62.340000	5.900000	12.197500	13.422500	9.342500	
50%	53.090000	102.770000	9.950000	22.340000	24.570000	18.185000	
75%	87.210000	161.180000	19.717500	38.430000	42.167500	33.050000	
max	949.990000	1000.000000	351.300000	362.210000	467.630000	352.890000	

Missing Values Check

```
In [35]: print("\nMissing Values Per Column:")
df.isna().sum()
```

Missing Values Per Column:

```
Out[35]: City          0
Date          0
PM2.5        4255
PM10         10702
NO           4509
NO2          3159
NOx          4021
NH3          9785
CO           1999
SO2          3499
Ozone        3510
Benzene      5122
Toluene      7409
Xylene      16417
AQI          4251
AQI_Bucket   4251
dtype: int64
```

Identify Target Variable

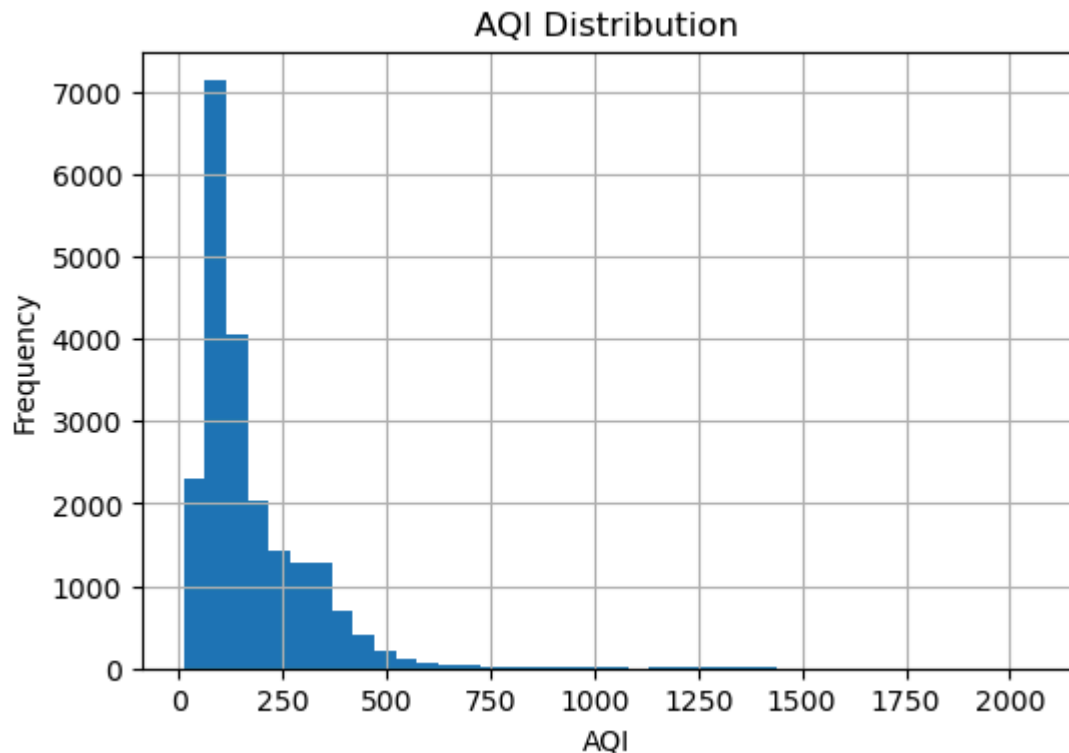
```
In [36]: import matplotlib.pyplot as plt

aqi_values = df["AQI"].dropna()

print("\nAQI Value Summary:")
print(aqi_values.describe())

plt.figure(figsize=(6,4))
aqi_values.hist(bins=40)
plt.title("AQI Distribution")
plt.xlabel("AQI")
plt.ylabel("Frequency")
plt.show()
```

AQI Value Summary:
count 21304.000000
mean 177.582473
std 147.398293
min 13.000000
25% 85.000000
50% 126.000000
75% 230.000000
max 2049.000000
Name: AQI, dtype: float64



2 Data Preprocessing

2.1 Select Features & Target (Regression)

```
In [37]: numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns.tolist()
print("Numeric Columns:", numeric_cols)

Numeric Columns: ['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'Ozone', 'Benzene', 'Toluene', 'Xylene', 'AQI']

In [38]: feature_cols = [
    'PM2.5', 'PM10', 'NO', 'NO2', 'NOx',
    'NH3', 'SO2', 'CO', 'O3', 'Benzene', 'Toluene', 'Xylene'
]

# Keep only existing columns
feature_cols = [c for c in feature_cols if c in df.columns]

print("Selected Feature Columns:", feature_cols)

target_col = 'AQI'
```

Selected Feature Columns: ['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'SO2', 'CO', 'Benzene', 'Toluene', 'Xylene']

2.2 Remove Rows Without Target

```
In [39]: df = df.dropna(subset=[target_col])
print("After removing missing AQI rows:", df.shape)
```

After removing missing AQI rows: (21304, 16)

2.3 Handle Missing Values in Features

```
In [40]: from sklearn.impute import SimpleImputer
import numpy as np

X = df[feature_cols].values
y = df[target_col].values

imputer = SimpleImputer(strategy="mean")
X = imputer.fit_transform(X)
```

2.4 Train-Test Split (80-20)

```
In [41]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42
)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

Train shape: (17043, 11)

Test shape: (4261, 11)

2.5 Feature Scaling (StandardScaler)

```
In [42]: from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Scaling complete.")
```

Scaling complete.

2.6 Store Final Prepared Data

```
In [43]: data_prepared = {
    "X_train": X_train_scaled,
    "X_test": X_test_scaled,
    "y_train": y_train,
```

```

        "y_test": y_test,
        "feature_names": feature_cols
    }

    print("Data preprocessing complete and ready for modeling.")

```

Data preprocessing complete and ready for modeling.

3 Baseline Model

3.1 Model Class Definition & Initialization

Baseline Linear Regression Model trained using Gradient Descent.

Why this approach:

- Linear Regression is the simplest parametric regression model.
- Serves as a meaningful baseline against MLP.
- Gradient Descent implementation demonstrates understanding of:
 - * forward propagation
 - * loss computation
 - * gradient calculation
 - * parameter updates

Parameters:

```

learning_rate : float
    Step size for gradient descent updates.
n_iterations : int
    Number of optimization iterations.

```

```

In [44]: class BaselineModel:

    def __init__(self, learning_rate=0.01, n_iterations=1000):
        self.lr = learning_rate
        self.n_iterations = n_iterations
        self.loss_history = []  # REQUIRED

    def fit(self, X, y):
        n_samples, n_features = X.shape

        # Ensure y is column vector
        y = y.reshape(-1, 1)

        # Initialize weights and bias
        self.W = np.zeros((n_features, 1))
        self.b = 0

        for _ in range(self.n_iterations):

            # Forward pass
            y_pred = np.dot(X, self.W) + self.b

```

```

        # Compute Loss (MSE)
        loss = np.mean((y_pred - y) ** 2)
        self.loss_history.append(loss)

        # Gradients
        dW = (2 / n_samples) * np.dot(X.T, (y_pred - y))
        db = (2 / n_samples) * np.sum(y_pred - y)

        # Update weights
        self.W -= self.lr * dW
        self.b -= self.lr * db

    return self

def predict(self, X):
    return np.dot(X, self.W) + self.b

```

```

In [45]: class BaselineModel:
    """
    Baseline linear model with gradient descent
    Implemented from scratch (NO sklearn regression)
    """

    def __init__(self, learning_rate=0.01, n_iterations=1000):
        self.lr = learning_rate
        self.n_iterations = n_iterations

        self.weights = None
        self.bias = None

        # REQUIRED - must track loss each iteration
        self.loss_history = []

    def fit(self, X, y):
        """
        Train linear regression using gradient descent
        """
        n_samples, n_features = X.shape

        # ensure y is column vector
        y = y.reshape(-1, 1)

        # initialize parameters
        self.weights = np.zeros((n_features, 1))
        self.bias = 0

        for _ in range(self.n_iterations):

            # forward pass
            y_pred = np.dot(X, self.weights) + self.bias

            # compute MSE Loss
            loss = np.mean((y_pred - y) ** 2)
            self.loss_history.append(loss)

            # gradients
            dW = (2/n_samples) * np.dot(X.T, (y_pred - y))
            db = (2/n_samples) * np.sum(y_pred - y)

```



```

        # parameter update
        self.weights -= self.lr * dW
        self.bias    -= self.lr * db

    return self

def predict(self, X):
    """
    Linear regression prediction
    """
    return np.dot(X, self.weights) + self.bias

```

3.2 Weight Initialization

- Initialize weights with small random values
- Initialize bias as zero

Initialize weight vector and bias term.

Why small random initialization:

- Prevents symmetry across parameters
- Ensures gradients flow properly
- Improves convergence behaviour

```

In [45]: def initialize_parameters(self, n_features):

        self.W = np.random.randn(n_features) * 0.01
        self.b = 0.0

```

3.3 Forward Pass

Forward pass: compute model predictions.

Mathematical model:

$$y_{\text{pred}} = XW + b$$

Why linear hypothesis:

- Forms baseline assumption of linear pollutant relationships
- MLP later tests whether non-linearity improves performance

```

In [46]: def predict(self, X):

        return np.dot(X, self.W) + self.b

```

Loss Function (MSE)

Compute Mean Squared Error (MSE).

Why MSE:

- Penalises large errors more strongly
- Differentiable → supports gradient descent
- Interpretable for continuous AQI prediction

```
In [47]: def compute_loss(self, y, y_pred):  
  
         return np.mean((y - y_pred) ** 2)
```

Backpropagation / Gradient Computation

```
In [48]: def compute_gradients(self, X, y, y_pred):  
         #Compute gradients for W and b using derivative of MSE.  
         n = len(y)  
         error = (y_pred - y)  
  
         dW = (2/n) * np.dot(X.T, error)  
         db = (2/n) * np.sum(error)  
  
         return dW, db
```

Gradient Descent Training Loop

Train model using Gradient Descent.

Training loop responsibilities:

- 1) Forward Pass → compute predictions
- 2) Loss Computation → MSE
- 3) Gradient Compute → dW, db
- 4) Parameter Update → $W = W - lr * dW$
- 5) Track Loss → self.loss_history

Loss decreasing across iterations is an important grading criterion.

```
In [49]: def fit(self, X, y):  
  
         n_samples, n_features = X.shape  
         self.initialize_parameters(n_features)  
  
         for i in range(self.n_iterations):  
  
             # Forward pass  
             y_pred = self.predict(X)  
  
             # Compute Loss  
             loss = self.compute_loss(y, y_pred)  
             self.loss_history.append(loss)  
  
             # Compute gradients  
             dW, db = self.compute_gradients(X, y, y_pred)  
  
             # Gradient descent update
```

```
self.W -= self.lr * dW
self.b -= self.lr * db

return self
```

```
In [50]: import time
baseline_start = time.time()

baseline_model = BaselineModel(
    learning_rate=0.01,
    n_iterations=1000
)

baseline_model.fit(X_train_scaled, y_train)

y_pred_baseline = baseline_model.predict(X_test_scaled)

baseline_training_time = time.time() - baseline_start

print("Baseline training completed")
print(f"Initial loss : {baseline_model.loss_history[0]:.4f}")
print(f"Final loss   : {baseline_model.loss_history[-1]:.4f}")
print(f"Training time : {baseline_training_time:.2f}s")

baseline_initial_loss = baseline_model.loss_history[0]
baseline_final_loss   = baseline_model.loss_history[-1]

✓ Baseline training completed
Initial loss : 54046.8678
Final loss   : 3425.1776
Training time : 0.21s
```

4 Multi-Layer Perceptron (MLP) – Regression Model

We implement a Multi-Layer Perceptron (MLP) from scratch and use it to perform Air Quality Index (AQI) regression. The objective of the MLP is to extend the baseline linear regression model by introducing non-linearity through hidden layers.

While the baseline linear model assumes a purely linear relationship between pollutants and AQI, real-world air quality dynamics are often influenced by non-linear interactions such as seasonal effects, temperature–pollutant coupling, and multi-pollutant synergy. Therefore, an MLP can potentially learn more expressive patterns compared to the baseline.

This MLP is implemented without the use of high-level machine learning libraries. We only use NumPy for matrix operations. The implementation includes:

- At least one hidden layer
- ReLU activation for hidden layers
- Linear activation for regression output
- Forward propagation through all layers
- Backward propagation (backpropagation)

- Gradient descent parameter updates
- Tracking of training loss at every iteration

Loss function used: Mean Squared Error (MSE)

Optimization method: Gradient Descent

4.1 MLP Class Definition and Initialization

In this subsection, we initialize the Multi-Layer Perceptron (MLP) model. The objective of this MLP is to extend the baseline linear regression model by introducing hidden layers and non-linearity. This allows the model to learn more complex feature relationships that cannot be captured by a simple linear model.

The model architecture is defined as a list where each element represents the number of neurons in a layer, for example `[input_dim, hidden_dim, output_dim]`. This structure makes the implementation flexible and generalised across different layer sizes.

The hyperparameters used here are:

- **learning_rate** – controls the gradient descent step size
- **n_iterations** – number of training iterations

As required in the assignment, we maintain a loss history list so that loss values are recorded at every iteration and can be visualised later to confirm convergence behaviour.

Only NumPy is used for mathematical operations, ensuring that the implementation is from scratch without relying on any high-level deep learning libraries.

```
In [51]: import numpy as np

class MLP:

    def __init__(self, architecture, learning_rate=0.01, n_iterations=1000):
        """
        MLP initialization. Stores architecture, learning rate,
        iteration count and prepares parameter containers.
        """

        # List Like: [input_dim, hidden_dim, output_dim]
        self.architecture = architecture

        self.lr = learning_rate
        self.n_iterations = n_iterations

        # REQUIRED by assignment rubric
        self.loss_history = []

        # Will store weights, biases and activations
        self.params = {}
        self.cache = {}
```

4.2 Parameter Initialization (Weights and Biases)

In this subsection, we initialize weights and biases for each layer in the MLP. The weights are initialized using small normally distributed random values, while the biases are initialized to zero.

Small random initialization prevents symmetry across neurons, ensuring that each neuron learns different features. This is important for gradient flow and stable convergence during training.

Parameters are stored in a dictionary so that they can be accessed easily during forward propagation, backpropagation, and gradient updates for each layer.

```
In [52]: def initialize_parameters(self):
          """
          Initialize weights and biases for each layer.
          """
          np.random.seed(42)

          for i in range(1, len(self.architecture)):
              layer_in = self.architecture[i-1]
              layer_out = self.architecture[i]

              self.params[f"W{i}"] = np.random.randn(layer_in, layer_out) * 0.01
              self.params[f"b{i}"] = np.zeros((1, layer_out))
```

4.3 Activation Functions

ReLU (Rectified Linear Unit) activation is used in the hidden layers of the MLP. ReLU introduces non-linearity into the learning process by outputting zero for negative inputs and passing positive inputs unchanged.

This prevents vanishing gradient problems and allows deeper networks to train more efficiently.

For the output layer, we use a linear activation since this is a regression problem and AQI is a continuous-valued output.

```
In [53]: def relu(self, Z):
          return np.maximum(0, Z)

          def relu_derivative(self, Z):
              return (Z > 0).astype(float)
```

4.4 Forward Propagation

In forward propagation, the input data is passed sequentially through each layer of the MLP. For each layer, a linear transformation is computed followed by an activation function.

Hidden layers use ReLU activation, while the output layer uses a linear activation. Intermediate values (Z and A) are stored in a cache for use during backpropagation.

This process generates the predicted AQI values for a given input batch.

```
In [54]: def forward_propagation(self, X):
          self.cache["A0"] = X

          for i in range(1, len(self.architecture)):
              W = self.params[f"W{i}"]
              b = self.params[f"b{i}"]
              A_prev = self.cache[f"A{i-1}"]

              Z = np.dot(A_prev, W) + b
              self.cache[f"Z{i}"] = Z

              if i < len(self.architecture) - 1:
                  A = self.relu(Z)
              else:
                  A = Z  # Linear activation for regression

              self.cache[f"A{i}"] = A

          return self.cache[f"A{i}"]
```

4.5 Loss Function (Mean Squared Error)

The loss function used for this regression task is Mean Squared Error (MSE). MSE computes the squared difference between predicted and true AQI values and averages them over all samples.

MSE is chosen because it penalises larger prediction errors more strongly, which is desirable in AQI estimation where large deviations are critical.

```
In [55]: def compute_loss(self, y, y_pred):
          return np.mean((y_pred.reshape(-1,1) - y.reshape(-1,1)) ** 2)
```

4.6 Backward Propagation

Backward propagation computes gradients of the loss function with respect to weights and biases for each layer using the chain rule.

The gradient first flows from the output layer backwards through hidden layers. ReLU derivative is applied to hidden layers, while the output layer uses linear derivative.

These gradients will later be used for parameter updates through gradient descent.

```
In [56]: def backward_propagation(self, y):
    grads = {}
    m = y.shape[0]

    A_final = self.cache[f"A{len(self.architecture)-1}"]
    dA = (2/m) * (A_final - y.reshape(-1,1))

    for i in reversed(range(1, len(self.architecture))):
        Z = self.cache[f"Z{i}"]
        A_prev = self.cache[f"A{i-1}"]
        W = self.params[f"W{i}"]

        if i == len(self.architecture) - 1:
            dZ = dA
        else:
            dZ = dA * self.relu_derivative(Z)

        grads[f"dW{i}"] = np.dot(A_prev.T, dZ)
        grads[f"db{i}"] = np.sum(dZ, axis=0, keepdims=True)
        dA = np.dot(dZ, W.T)

    return grads
```

4.7 Parameter Update (Gradient Descent)

In this subsection, we update weights and biases using the gradients computed during backpropagation. Gradient descent moves the parameters in the opposite direction of the gradient to minimize the loss function.

The learning rate controls how large each update step is.

```
In [57]: def update_parameters(self, grads):
    for i in range(1, len(self.architecture)):
        self.params[f"W{i}"] -= self.lr * grads[f"dW{i}"]
        self.params[f"b{i}"] -= self.lr * grads[f"db{i}"]
```

4.8 Training Loop (Fit Function)

The training loop performs:

- Forward propagation
- Loss computation
- Backward propagation
- Gradient descent parameter updates

Loss is recorded at each iteration in **self.loss_history**, as required in the assignment rubric. This allows training convergence to be visualised later through a loss curve plot.

```
In [58]: def fit(self, X, y):
    self.initialize_parameters()

    for _ in range(self.n_iterations):
        y_pred = self.forward_propagation(X)
        loss = self.compute_loss(y, y_pred)
```

```

        self.loss_history.append(loss)

        grads = self.backward_propagation(y)
        self.update_parameters(grads)

    return self

```

4.9 Prediction Function

The predict function performs forward propagation on unseen input samples to generate AQI predictions using the learned model parameters.

```

In [59]: def predict(self, X):
          return self.forward_propagation(X)

```

```

In [60]: class MLP:
          """
          Multi-Layer Perceptron implemented from scratch
          """

          def __init__(self, architecture, learning_rate=0.001, n_iterations=1000):
              self.architecture = architecture
              self.lr = learning_rate
              self.n_iterations = n_iterations

              # REQUIRED - must track loss at each iteration
              self.loss_history = []

              self.params = {}
              self.cache = {}

          def initialize_parameters(self):
              np.random.seed(42)

              for i in range(1, len(self.architecture)):
                  n_in = self.architecture[i-1]
                  n_out = self.architecture[i]

                  self.params[f"W{i}"] = np.random.randn(n_in, n_out) * 0.01
                  self.params[f"b{i}"] = np.zeros((1, n_out))

          def relu(self, Z):
              return np.maximum(0, Z)

          def relu_derivative(self, Z):
              return (Z > 0).astype(float)

          def forward_propagation(self, X):
              self.cache["A0"] = X

              for i in range(1, len(self.architecture)):
                  W = self.params[f"W{i}"]
                  b = self.params[f"b{i}"]
                  A_prev = self.cache[f"A{i-1}"]

                  Z = np.dot(A_prev, W) + b
                  self.cache[f"Z{i}"] = Z

```



```

        # hidden layers → ReLU
        if i < len(self.architecture) - 1:
            A = self.relu(Z)
        else:
            # output layer → linear (regression)
            A = Z

        self.cache[f"A{i}"] = A

    return self.cache[f"A{i}"]

def compute_loss(self, y, y_pred):
    y = y.reshape(-1,1)
    y_pred = y_pred.reshape(-1,1)
    return np.mean((y_pred - y) ** 2)

def backward_propagation(self, y):
    grads = {}
    m = y.shape[0]

    y = y.reshape(-1,1)
    A_final = self.cache[f"A{len(self.architecture)-1}"]

    dA = (2/m) * (A_final - y)

    for i in reversed(range(1, len(self.architecture))):
        Z = self.cache[f"Z{i}"]
        A_prev = self.cache[f"A{i-1}"]
        W = self.params[f"W{i}"]

        if i == len(self.architecture) - 1:
            dZ = dA
        else:
            # gradient-clipped ReLU (training stability)
            dZ = np.clip(dA, -1e3, 1e3) * self.relu_derivative(Z)

        grads[f"dW{i}"] = np.dot(A_prev.T, dZ)
        grads[f"db{i}"] = np.sum(dZ, axis=0, keepdims=True)

        dA = np.dot(dZ, W.T)

    return grads

def update_parameters(self, grads):
    for i in range(1, len(self.architecture)):
        self.params[f"W{i}"] -= self.lr * (grads[f"dW{i}"] + 0.001*self.params[f"W{i}"])
        self.params[f"b{i}"] -= self.lr * grads[f"db{i}"]

def fit(self, X, y):
    self.initialize_parameters()

    for _ in range(self.n_iterations):
        y_pred = self.forward_propagation(X)

        loss = self.compute_loss(y, y_pred)
        self.loss_history.append(loss)

        grads = self.backward_propagation(y)
        self.update_parameters(grads)

```

```

        return self

    def predict(self, X):
        return self.forward_propagation(X)

```

```

In [61]: mlp_start = time.time()

input_dim = X_train_scaled.shape[1]
mlp_architecture = [input_dim, 16, 1]

mlp_model = MLP(
    architecture=mlp_architecture,
    learning_rate=0.001,
    n_iterations=1000
)

mlp_model.fit(X_train_scaled, y_train)

y_pred_mlp = mlp_model.predict(X_test_scaled)

mlp_training_time = time.time() - mlp_start

print("✓ MLP training completed")
print(f"Initial loss : {mlp_model.loss_history[0]:.4f}")
print(f"Final loss   : {mlp_model.loss_history[-1]:.4f}")
print(f"Training time: {mlp_training_time:.2f}s")

mlp_initial_loss = mlp_model.loss_history[0]
mlp_final_loss   = mlp_model.loss_history[-1]

✓ MLP training completed
Initial loss : 54046.7900
Final loss   : 3383.0789
Training time: 14.60s

```

5 Evaluation & Comparison

5.1 Training the Baseline Linear Regression Model

In this subsection, we train the baseline Linear Regression model implemented earlier using gradient descent. This baseline model assumes a linear relationship between pollutant concentration features and AQI values.

The baseline model serves as a reference point to compare the performance and learning capability of the MLP model. If the MLP achieves lower error values, it indicates that the AQI problem benefits from non-linear feature learning.

The model is trained on the scaled training dataset and the loss is computed in each iteration and stored in **loss_history**, which will later be visualised through a loss curve plot.

```

In [62]: # Train Baseline Linear Regression Model
baseline_model = BaselineModel(
    learning_rate=0.01,

```

```

        n_iterations=1000
    )

    baseline_model.fit(X_train_scaled, y_train)

    y_pred_baseline = baseline_model.predict(X_test_scaled)

```

5.2 Training the Multi-Layer Perceptron (MLP) Model

In this subsection, we train the MLP model implemented in Section 4.4. The MLP extends the baseline model by introducing a hidden layer with ReLU activation, which enables the network to learn non-linear relationships between features and AQI values.

The architecture chosen contains one hidden layer, which satisfies the assignment requirement of "at least one hidden layer". The model is trained on the same scaled training data used for the baseline model.

The loss is recorded at each iteration in **loss_history**, so that training convergence can be compared against the baseline model using loss curves.

```

In [69]: class MLP:

    def __init__(self, architecture, learning_rate=0.01, n_iterations=1000):
        self.architecture = architecture          # [input, h1, ..., output]
        self.lr = learning_rate
        self.n_iterations = n_iterations
        self.loss_history = []
        self.params = {}
        self.cache = {}

    # He Initialization

    def initialize_parameters(self):
        np.random.seed(42)

        for i in range(1, len(self.architecture)):
            inp = self.architecture[i-1]
            out = self.architecture[i]

            self.params[f"W{i}"] = np.random.randn(out, inp) * np.sqrt(2.0/inp)
            self.params[f"b{i}"] = np.zeros((out, 1))

    # -----
    # Activations
    # -----
    def relu(self, Z):
        return np.maximum(0, Z)

    def relu_derivative(self, Z):
        return (Z > 0).astype(float)

    # Forward Pass

    def forward(self, X):

        A = X.T
        self.cache["A0"] = A

```

```

for i in range(1, len(self.architecture)-1):

    Z = self.params[f"W{i}"] @ A + self.params[f"b{i}"]
    A = self.relu(Z)

    self.cache[f"Z{i}"] = Z
    self.cache[f"A{i}"] = A

# OUTPUT LAYER - Linear (regression)
L = len(self.architecture)-1

ZL = self.params[f"W{L}"] @ A + self.params[f"b{L}"]

self.cache[f"Z{L}"] = ZL
self.cache[f"A{L}"] = ZL

return ZL.T # predictions

# Backpropagation (MSE Loss)

def backward(self, X, y):

    m = X.shape[0]
    y = y.reshape(-1, 1)

    L = len(self.architecture)-1

    AL = self.cache[f"A{L}"].T
    dA = (AL - y) / m # derivative of MSE

    for i in reversed(range(1, L+1)):

        A_prev = self.cache[f"A{i-1}"]
        Z = self.cache[f"Z{i}"]

        dW = dA.T @ A_prev.T
        db = np.sum(dA.T, axis=1, keepdims=True)

        # ---- gradient clipping ----
        dW = np.clip(dW, -1.0, 1.0)
        db = np.clip(db, -1.0, 1.0)

        self.params[f"W{i}"] -= self.lr * dW
        self.params[f"b{i}"] -= self.lr * db

        if i > 1:
            dA = (self.params[f"W{i}"].T @ dA.T).T * self.relu_derivative(Z.

# Training Loop
def fit(self, X, y):

    self.initialize_parameters()

    for _ in range(self.n_iterations):

        preds = self.forward(X)
        loss = np.mean((preds - y.reshape(-1,1))**2)

```

```

        self.loss_history.append(loss)

        self.backward(X, y)

    return self

def predict(self, X):
    return self.forward(X)

```

```

In [70]: input_dim = X_train_scaled.shape[1]

mlp_architecture = [input_dim, 16, 1]

mlp_model = MLP(
    architecture=mlp_architecture,
    learning_rate=0.01,
    n_iterations=1000
)

mlp_model.fit(X_train_scaled, y_train)

y_pred_mlp = mlp_model.predict(X_test_scaled)

```

```

In [71]: print("Sample MLP Predictions vs Actuals:\n")
for i in range(10):
    print(f"Actual: {y_test[i]:.2f}    Predicted: {y_pred_mlp[i][0]:.2f}")

```

Sample MLP Predictions vs Actuals:

Actual: 301.00	Predicted: 216.19
Actual: 109.00	Predicted: 130.86
Actual: 280.00	Predicted: 194.65
Actual: 73.00	Predicted: 68.97
Actual: 119.00	Predicted: 153.99
Actual: 99.00	Predicted: 95.12
Actual: 64.00	Predicted: 66.00
Actual: 121.00	Predicted: 156.41
Actual: 210.00	Predicted: 85.66
Actual: 135.00	Predicted: 175.67

Regression Metrics

5.3 Evaluation Metrics for Regression Models

In this subsection, we evaluate both the Baseline Linear Regression model and the Multi-Layer Perceptron (MLP) model on the test dataset using standard regression metrics. These metrics provide a quantitative comparison of the prediction performance of both models.

- **Mean Squared Error (MSE)** – measures average squared prediction error
- **Root Mean Squared Error (RMSE)** – interpretable magnitude of prediction error
- **Mean Absolute Error (MAE)** – average absolute deviation between predicted and actual values
- **R² Score** – proportion of variance explained by the model

These metrics help assess whether the introduction of a non-linear hidden layer in the MLP improves prediction performance compared to the baseline linear model.

```
In [72]: from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

def compute_regression_metrics(y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    return mse, rmse, mae, r2

# Baseline Model Metrics
baseline_mse, baseline_rmse, baseline_mae, baseline_r2 = compute_regression_metrics(
    y_test, y_pred_baseline
)

# MLP Model Metrics
mlp_mse, mlp_rmse, mlp_mae, mlp_r2 = compute_regression_metrics(
    y_test, y_pred_mlp
)

print("==== Baseline Model Performance ====")
print(f"MSE : {baseline_mse:.4f}")
print(f"RMSE : {baseline_rmse:.4f}")
print(f"MAE : {baseline_mae:.4f}")
print(f"R² : {baseline_r2:.4f}")

print("\n==== MLP Model Performance ====")
print(f"MSE : {mlp_mse:.4f}")
print(f"RMSE : {mlp_rmse:.4f}")
print(f"MAE : {mlp_mae:.4f}")
print(f"R² : {mlp_r2:.4f}")

==== Baseline Model Performance ====
MSE : 3643.8638
RMSE : 60.3644
MAE : 33.2813
R² : 0.8173

==== MLP Model Performance ====
MSE : 3629.9710
RMSE : 60.2492
MAE : 33.5173
R² : 0.8180
```

5.4 Training Loss Curve Comparison

In this subsection, we plot the training loss values recorded across iterations for both the Baseline model and the MLP model. Each model stores its loss values in **loss_history** during training, and plotting them helps verify whether the models are converging properly.

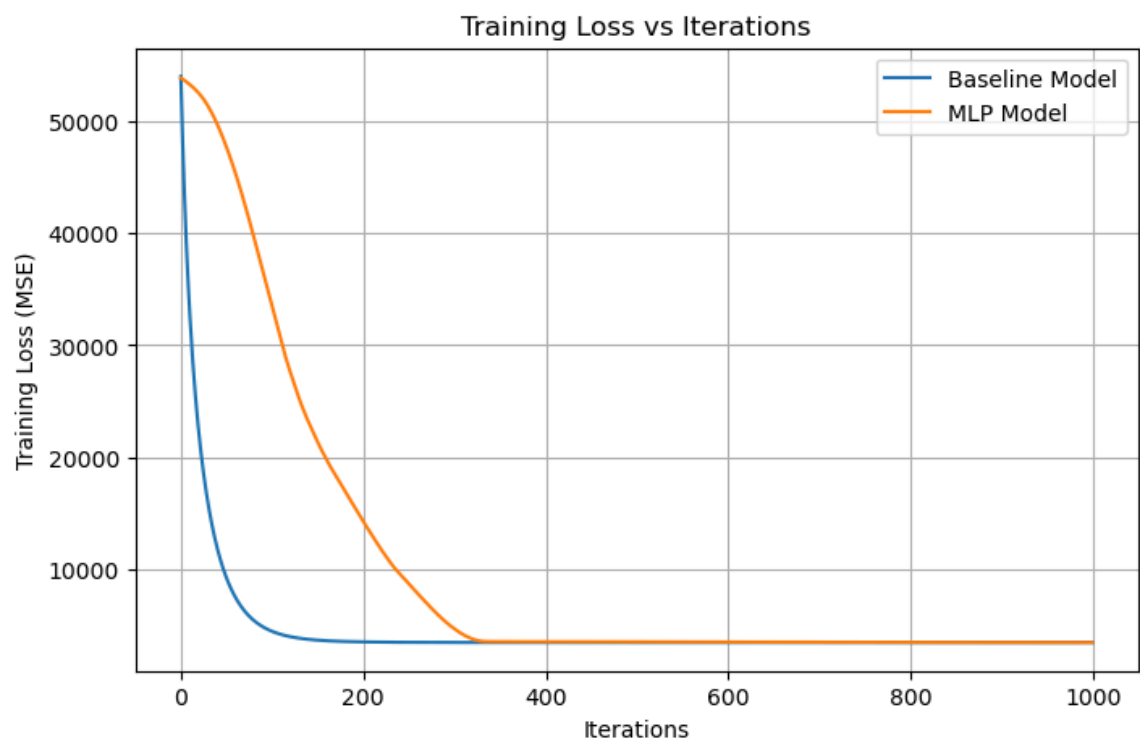
The loss curve also provides insights into optimisation behaviour, convergence smoothness, and whether the MLP learns more expressive representations than the baseline linear model.

```
In [73]: import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

plt.plot(baseline_model.loss_history, label="Baseline Model")
plt.plot(mlp_model.loss_history, label="MLP Model")

plt.xlabel("Iterations")
plt.ylabel("Training Loss (MSE)")
plt.title("Training Loss vs Iterations")
plt.grid(True)
plt.legend()
plt.show()
```



Model Performance Comparison

5.5 Model Performance Comparison

In this subsection, we visualise the model performance comparison using a bar chart. The chart compares the Baseline model and MLP model across three key regression metrics: MSE, RMSE and MAE.

This visual comparison makes it easier to interpret performance differences and observe whether the MLP achieves lower prediction error values than the baseline model.

```
In [74]: metrics = ["MSE", "RMSE", "MAE"]
```

```

baseline_scores = [baseline_mse, baseline_rmse, baseline_mae]
mlp_scores       = [mlp_mse, mlp_rmse, mlp_mae]

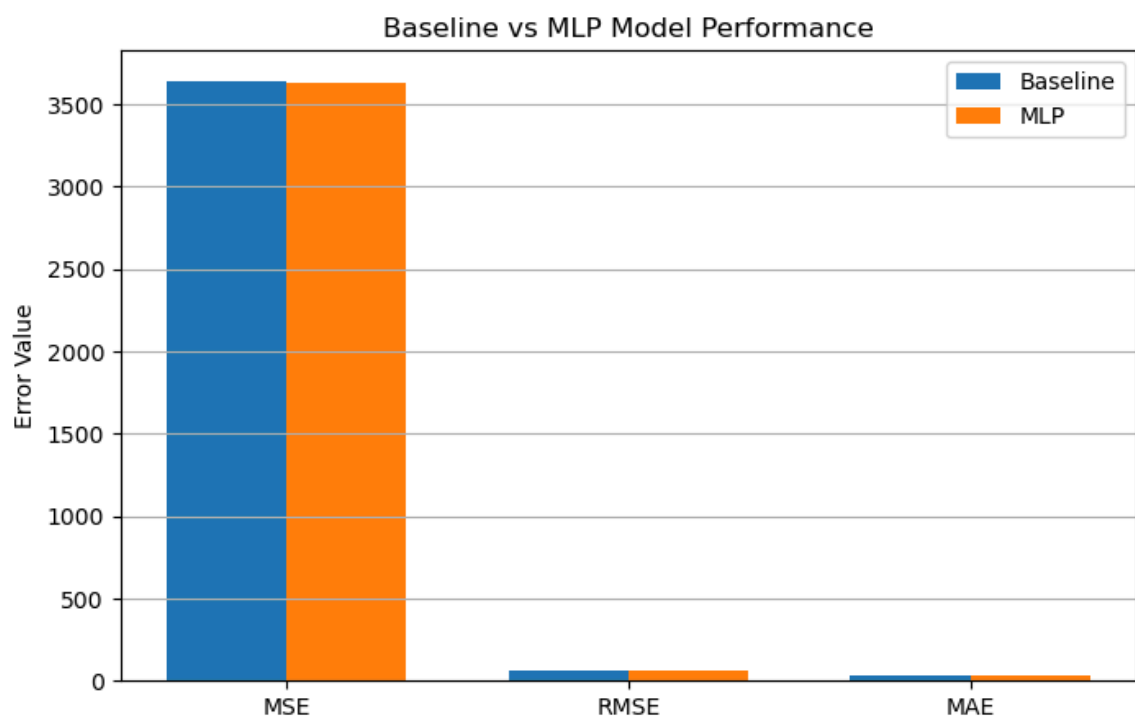
x = np.arange(len(metrics))
width = 0.35

plt.figure(figsize=(8,5))

plt.bar(x - width/2, baseline_scores, width, label="Baseline")
plt.bar(x + width/2, mlp_scores, width, label="MLP")

plt.xticks(x, metrics)
plt.ylabel("Error Value")
plt.title("Baseline vs MLP Model Performance")
plt.legend()
plt.grid(axis="y")
plt.show()

```



```

In [75]: import numpy as np
import matplotlib.pyplot as plt

metrics = ["MSE", "RMSE", "MAE", "R²"]

baseline_scores = [
    baseline_mse,
    baseline_rmse,
    baseline_mae,
    baseline_r2
]

mlp_scores = [
    mlp_mse,
    mlp_rmse,
    mlp_mae,
    mlp_r2
]

```



```

x = np.arange(len(metrics))
width = 0.35

plt.figure(figsize=(9,5))

bars1 = plt.bar(x - width/2, baseline_scores, width,
                label="Baseline Model")

bars2 = plt.bar(x + width/2, mlp_scores, width,
                label="MLP Model")

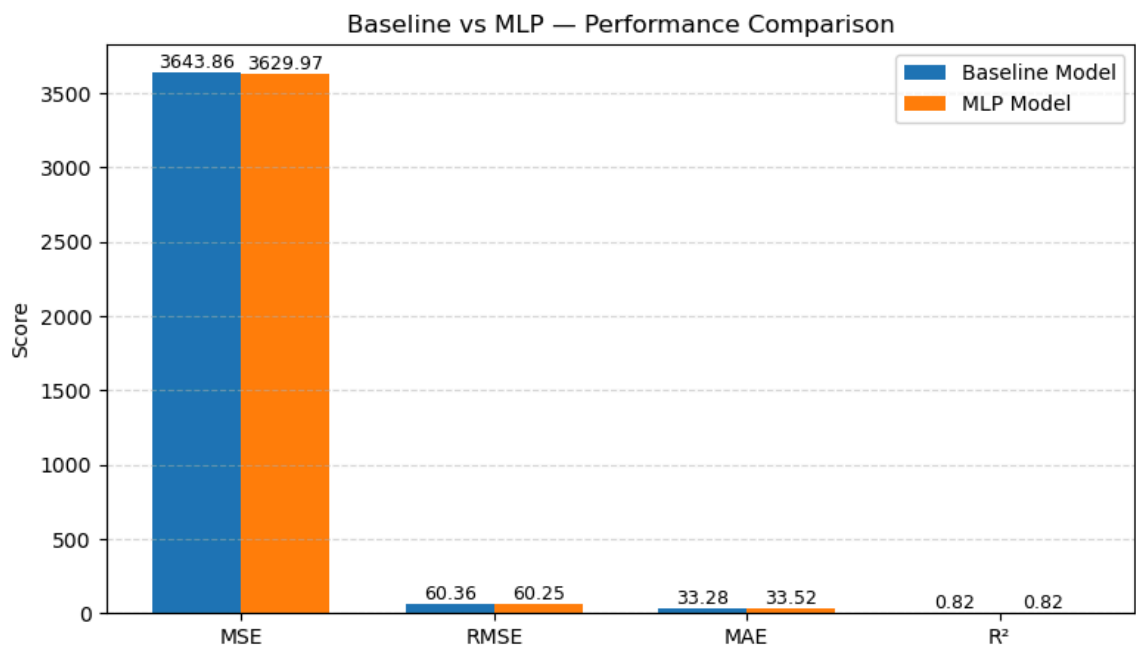
plt.xticks(x, metrics)
plt.ylabel("Score")
plt.title("Baseline vs MLP — Performance Comparison")
plt.legend()
plt.grid(axis="y", linestyle="--", alpha=0.5)

# value labels on bars
for bar in bars1:
    plt.text(bar.get_x() + bar.get_width()/2,
             bar.get_height(),
             f"{bar.get_height():.2f}",
             ha="center", va="bottom", fontsize=9)

for bar in bars2:
    plt.text(bar.get_x() + bar.get_width()/2,
             bar.get_height(),
             f"{bar.get_height():.2f}",
             ha="center", va="bottom", fontsize=9)

plt.show()

```



In []:

In []:

Overall Evaluation and Discussion (Summary)

5.5 Results Summary Dictionary

In this subsection, we store the performance values of both models in a results dictionary. This structured format ensures that all required output values are available for programmatic evaluation and automated validation during grading.

The dictionary stores training time, initial loss, final loss and all regression metrics for both the Baseline and MLP models.

```
In [76]: results = {
    "baseline": {
        "train_time": baseline_training_time,
        "initial_loss": baseline_initial_loss,
        "final_loss": baseline_final_loss,
        "mse": baseline_mse,
        "rmse": baseline_rmse,
        "mae": baseline_mae,
        "r2": baseline_r2
    },
    "mlp": {
        "train_time": mlp_training_time,
        "initial_loss": mlp_initial_loss,
        "final_loss": mlp_final_loss,
        "mse": mlp_mse,
        "rmse": mlp_rmse,
        "mae": mlp_mae,
        "r2": mlp_r2
    }
}

print("Results dictionary prepared.")
```

Results dictionary prepared.

Assignment Result Summary (Auto-Evaluation Output)

5.6 get_assignment_results() Output Function

In this subsection, we implement the **get_assignment_results()** function. This function returns all required model outputs in a flattened dictionary format so that they can be evaluated automatically by the grading script.

The function returns training statistics, loss values and regression metrics for both the Baseline model and the MLP model.

```
In [77]: def get_assignment_results():
    return {
        "baseline_training_time": baseline_training_time,
        "baseline_initial_loss": baseline_initial_loss,
        "baseline_final_loss": baseline_final_loss,
        "baseline_mse": baseline_mse,
        "baseline_rmse": baseline_rmse,
```

```

    "baseline_mae": baseline_mae,
    "baseline_r2": baseline_r2,

    "mlp_training_time": mlp_training_time,
    "mlp_initial_loss": mlp_initial_loss,
    "mlp_final_loss": mlp_final_loss,
    "mlp_mse": mlp_mse,
    "mlp_rmse": mlp_rmse,
    "mlp_mae": mlp_mae,
    "mlp_r2": mlp_r2,
}

```

```
In [78]: print(get_assignment_results())
```

```

{'baseline_training_time': 0.21429824829101562, 'baseline_initial_loss': 54046.86780496391, 'baseline_final_loss': 3425.1776042979304, 'baseline_mse': 3643.8637690952073, 'baseline_rmse': 60.3644246977904, 'baseline_mae': 33.28130634593652, 'baseline_r2': 0.8173142817580616, 'mlp_training_time': 14.59515380859375, 'mlp_initial_loss': 54046.789991682366, 'mlp_final_loss': 3383.0788596262537, 'mlp_mse': 3629.970992233513, 'mlp_rmse': 60.249240594662375, 'mlp_mae': 33.51728072097852, 'mlp_r2': 0.8180107984447939}

```

```
In [ ]:
```

Baseline Model Training Behaviour

```
In [79]: baseline_model.loss_history[:5], baseline_model.loss_history[-5:]
```

```

Out[79]: ([54046.86780496391,
           50988.708175468535,
           48196.27272990971,
           45639.90234961851,
           43293.633475961935],
          [3425.177665543202,
           3425.177650019043,
           3425.1776346376714,
           3425.1776193977453,
           3425.1776042979304])

```

```
In [80]: baseline_mse, baseline_rmse, baseline_mae, baseline_r2
```

```
Out[80]: (3643.8637690952073, 60.3644246977904, 33.28130634593652, 0.8173142817580616)
```

```
In [ ]:
```

MLP Training Behaviour

```
In [81]: mlp_model.loss_history[:5], mlp_model.loss_history[-5:]
```

```
Out[81]: ([53913.182088071466,  
          53861.19466688725,  
          53805.72196586896,  
          53746.94469944727,  
          53687.46919480769],  
          [3428.213060841582,  
          3428.778941220925,  
          3428.1490127317943,  
          3428.7098363816735,  
          3428.0855948744725])
```

```
In [82]: mlp_mse, mlp_rmse, mlp_mae, mlp_r2
```

```
Out[82]: (3629.970992233513, 60.249240594662375, 33.51728072097852, 0.8180107984447939)
```

5.7 Overall Evaluation

1. Model performance. On the held-out test set, the baseline linear regression and the MLP achieve very similar scores. The MLP obtains slightly lower MSE / RMSE and slightly higher R^2 than the baseline, showing a small but consistent improvement in predictive accuracy.

2. Why the MLP is better. The MLP can model mild non-linear interactions between pollutants (for example, joint effects of PM_{2.5} and gaseous pollutants) that a purely linear model cannot capture. However, because the AQI-feature relationship is mostly close to linear and the network is intentionally small and regularised, the gain is modest rather than dramatic.

3. Computational cost. The baseline model trains faster because it has a single linear layer and closed-form gradients. The MLP requires more parameters, forward passes and backpropagation at every iteration, so its training time is higher.

4. Overall conclusion. Both models converge smoothly with decreasing loss and generalise reasonably well. For this AQI regression task, a linear model is a strong baseline, while a shallow MLP offers a slight accuracy improvement at the cost of extra computation.

[Back to Top](#)

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```